

6-PART JAVASCRIPT SERIES · SESSION 1 OF 6 · 8 HOURS

JavaScript Class 1

Variables · Operators · Loops — foundations, internals, memory & performance, not just syntax

{ } JavaScript

Where Class 1 Fits in the 6-Class Journey

01

Variables, Operators, Loops

Today — memory model, best practices

02

Functions & Scope

Closures, higher-order fns, this binding

03

Objects & Arrays

Prototypes, references, deep vs shallow copy

04

Asynchronous JS

Event loop, Promises, async/await internals

05

DOM & Browser APIs

Rendering, events, performance profiling

06

Patterns & Project Build

Design patterns, debugging, capstone

Prerequisite Check & How This Class Is Different

- **You've completed HTML basics — good. Today we build the logic layer.**
- **This is NOT a syntax-only class. For every concept we cover four layers:**
 1. Concept — what it is and why it exists
 2. Best practice — the idiomatic, professional way to use it
 3. Memory behavior — what happens in the engine (stack/heap, GC)
 4. Code — a runnable example you should type yourself
- **Have Chrome DevTools (or Node.js) open — we will run every example live.**

Session Agenda (8 Hours)

- | | |
|------|---|
| Hr 1 | JS engine overview, execution context, call stack, memory heap |
| Hr 2 | Variables: var / let / const, scope, hoisting, TDZ |
| Hr 3 | Data types: primitives vs reference, stack vs heap allocation |
| Hr 4 | Memory management, garbage collection, common leak patterns |
| Hr 5 | Operators: arithmetic, comparison, logical, modern (??, ?.) |
| Hr 6 | Loops: for, while, do-while, for...of, for...in, break/continue |
| Hr 7 | Array iteration methods: forEach, map, filter, reduce + perf |
| Hr 8 | Best-practices recap, live debugging lab, Q&A, homework |

01

How JavaScript Runs

A 10-minute mental model of the engine before we touch a single variable.

The JavaScript Engine, In Brief

- JavaScript is single-threaded — one call stack, one thing executes at a time.
- The engine (V8 in Chrome/Node, SpiderMonkey in Firefox) has two memory regions you must know: the Call Stack and the Heap.
- Call Stack: stores execution contexts — fixed-size, primitive values, function calls.
- Heap: a large, mostly unstructured region for objects, arrays, functions — dynamic size.
- Every variable you declare today lives in one of these two places — that placement is what determines copy behavior, performance, and memory leaks.

Call Stack vs Heap — The Mental Model

- Call stack = LIFO structure, very fast, limited size (stack overflow if too deep).
- Heap = large, slower, garbage-collected region.
- Primitives (number, string, boolean, null, undefined, symbol, bigint) → stack.
- Objects, arrays, functions → heap; stack holds only a reference (pointer) to them.
- This single fact explains copy-by-value vs copy-by-reference, which we'll see in Hour 3.

CALL STACK

```
let age = 28
```

```
let name = 'Sam'
```

```
isActive = true
```

HEAP

```
{ user: {...} }  
[1, 2, 3, 4]  
function greet() {}
```

02

Variables

Declaration, scope, hoisting, and how each keyword behaves in memory.

Why Variables Exist

- A variable is a named reference to a value stored in memory — it lets us label, reuse, and mutate data.
- JavaScript is dynamically typed: a variable's type is determined at runtime, and can change.
- Three declaration keywords exist today: `var` (legacy, function-scoped), `let` (block-scoped, reassignable), `const` (block-scoped, not reassignable).
- Choosing the right keyword is a best practice decision, not just syntax — it affects scope leaks, hoisting bugs, and readability.

var — Function-Scoped & Hoisted

- Scope: function-level, not block-level — leaks out of if/for blocks.
- Hoisted to the top of its function and initialized with undefined.
- Can be redeclared and reassigned freely — a common source of bugs.
- Attaches to the global object (window/globalThis) when declared globally.
- Best practice: avoid var in new code; kept here only for legacy/interview knowledge.

```
function demo() {  
  if (true) {  
    var x = 10;  
  }  
  console.log(x); // 10 — leaked out of the block!  
}  
demo();  
  
console.log(y); // undefined, not an error  
var y = 5;      // hoisting in action
```

WHAT "HOISTED" MEANS

JS moves var and function declarations to the top of their scope before running — so y exists as undefined before its line executes.

let — Block-Scoped & Reassignable

- Scope: confined to the nearest `{ }` block — `if`, `for`, `while`, or a bare block.
- Hoisted, but NOT initialized — accessing before declaration throws (Temporal Dead Zone).
- Cannot be redeclared in the same scope — prevents accidental overwrites.
- Use `let` when a value must change over the variable's lifetime (counters, accumulators).
- Best practice: default to `let` only when reassignment is truly needed.

```
function demo() {  
  if (true) {  
    let x = 10;  
  }  
  console.log(x); // ReferenceError: x is not  
defined  
}  
  
let count = 0;  
count = count + 1; // reassignment is fine  
let count = 2;    // SyntaxError: already declared
```

const — Block-Scoped & Immutable Binding

- Same block scoping and TDZ (Temporal Dead Zone) behaviour as let.
- The BINDING is immutable — you cannot reassign the variable itself.
- Objects/arrays declared with const are still mutable internally (only the reference is locked).
- Best practice: default to const for everything; switch to let only when reassignment is required.
- Signals intent to readers: 'this reference never changes' — improves code readability.



```
const PI = 3.14159;  
PI = 3.14;           // TypeError: Assignment to  
                     constant  
  
const user = { name: "Sam" };  
user.name = "Alex"; // OK — mutating the object,  
                     not the binding  
user = {};           // TypeError: Assignment to  
                     constant
```

var vs let/const — The Decision in Practice

✗ Legacy Style

```
var total = 0;
for (var i = 0; i < items.length; i++) {
  var price = items[i].price;
  total += price;
}
console.log(i, price); // both leak out!
```

✓ Modern Style

```
let total = 0;
for (let i = 0; i < items.length; i++) {
  const price = items[i].price;
  total += price;
}
console.log(i); // ReferenceError — correctly scoped
```

WHY IT MATTERS

Block scoping with let/const prevents loop variables and temporaries from leaking into surrounding code — a leading cause of subtle bugs in closures and async callbacks (we'll revisit this in Class 2).

Hoisting & the Temporal Dead Zone (TDZ), Explained

- Hoisting = the engine scans a scope before execution and registers declarations.
- var declarations: hoisted AND initialized to undefined.
- let/const declarations: hoisted but left UNINITIALIZED — the zone between scope-start and the actual declaration line is the TDZ.
- Accessing a let/const variable inside the TDZ throws a ReferenceError — this is a safety feature, not a flaw.
- Best practice: declare variables as close as possible to first use — minimizes TDZ confusion.

```
{  
  // ---- TDZ starts ----  
  console.log(age);  
  // ReferenceError  
  
  let age = 28;  
  // ---- TDZ ends ----  
  console.log(age); // 28  
}
```

Naming Conventions & Readability Best Practices

- camelCase for variables and functions: `userAge`, `calculateTotal` — never `snake_case` or PascalCase for these.
- UPPER_SNAKE_CASE for true constants known at compile time: `MAX_RETRIES`, `API_BASE_URL`.
- PascalCase reserved for classes/constructors: `class ShoppingCart {}`.
- Names should describe intent, not type: `isLoading` not `loadingBool`; use verbs for functions (`getUser`, not `user2`).
- Avoid single-letter names outside tight loop counters (`i`, `j`) or math contexts.
- Avoid abbreviations that aren't universally understood (`usr`, `cnt`) — clarity beats brevity.

Variables — Quick Reference Card

var

Function-scoped · hoisted & initialized to undefined · avoid in new code

let

Block-scoped · hoisted, in TDZ until declared · reassignable

const

Block-scoped · binding is immutable · default choice for all variables

03

Data Types & Memory

Primitives vs reference types — and exactly how the engine stores each.

The 8 Data Types in JavaScript

- **Primitive types (7) — immutable, compared by value:**

number, string, boolean, undefined, null, symbol, bigint

- **Reference type (1) — mutable, compared by reference:**

object (this includes arrays, functions, dates, maps, sets, etc.)

- typeof operator inspects a value's type at runtime — but note: `typeof null === 'object'` (a famous historical bug kept for compatibility).
- Use `Array.isArray()` to check for arrays specifically — `typeof array === 'object'` is not enough.

Copy-by-Value vs Copy-by-Reference

- Assigning a primitive copies the actual value — two variables become fully independent.
- Assigning an object copies only the reference (a heap address) — both variables point to the same object.
- Mutating through one reference is visible through the other — a frequent source of bugs.
- This is exactly why `const` still allows `user.name = 'x'` — the reference doesn't change, only heap contents.
- Best practice: when you need an independent copy, use spread `{...obj}` / `[...arr]` or `structuredClone()` for deep copies.



```
let a = 10;
let b = a;
b = 20;
console.log(a); // 10 (independent)

let obj1 = { score: 10 };
let obj2 = obj1;
obj2.score = 20;
console.log(obj1.score); // 20 (shared!)
```

Shallow Copy vs Deep Copy in Practice

Understanding reference sharing tells you exactly when you need a copy — and which kind.



```
const original = { name: "Sam", address: { city: "Pune" } };

// Shallow copy — nested objects still shared!
const shallow = { ...original };
shallow.address.city = "Mumbai";
console.log(original.address.city); // "Mumbai" — leaked!

// Deep copy — fully independent
const deep = structuredClone(original);
deep.address.city = "Delhi";
console.log(original.address.city); // still "Mumbai"
```

Type Coercion — Know It, Then Avoid Relying On It

- JavaScript performs implicit type coercion with `==` and arithmetic/string operators.
- `'5' + 3 → '53'` (string concatenation wins); `'5' - 3 → 2` (numeric coercion wins).
- `[] == false → true`; `null == undefined → true`; these surprise even experienced developers.
- Best practice: always use `===` and `!==` (strict equality) — never rely on implicit coercion.
- Best practice: convert types explicitly — `Number(str)`, `String(num)`, `Boolean(val)` — so intent is visible in code.

04

Memory Management

Garbage collection, reference counting, and how to avoid leaks in real apps.

Garbage Collection: How JS Frees Memory

- You never manually free memory in JS — the Garbage Collector (GC) does it automatically.
- Core algorithm: Mark-and-Sweep. The GC starts from 'roots' (global object, active call stack) and marks everything reachable.
- Anything NOT reachable from a root is considered garbage and is swept (deallocated).
- 'Reachable' does not mean 'used' — a variable can be technically reachable yet functionally dead, causing a leak.
- V8 additionally uses generational GC: short-lived objects (young generation) are collected far more frequently and cheaply than long-lived ones (old generation).

Four Common Memory Leak Patterns

- **1. Accidental globals**

Forgetting `let/const/var` creates an implicit global that never gets collected.

- **2. Forgotten timers & intervals**

`setInterval` callbacks keep their closure scope alive until explicitly cleared.

- **3. Detached DOM references**

Keeping a JS reference to a removed DOM node prevents it from being garbage collected.

- **4. Uncleared event listeners / closures**

Listeners keep referencing outer variables even after the related object is no longer needed.

QUICK FIXES

1. Accidental globals

Enable 'use strict' — it throws instead of silently creating a global.

2. Forgotten timers

Store every interval/timeout ID and clear it in cleanup code.

3. Detached DOM refs

Null out variables once the DOM node they point to is removed.

4. Uncleared listeners

Call `removeEventListener`, or use `{ once: true }` for one-shot handlers.

Fixing a Real Memory Leak: Intervals

✗ Leaks Memory

```
function startPolling() {  
  setInterval(() => {  
    fetchData(); // keeps 'this' scope alive forever  
  }, 1000);  
}  
startPolling(); // no way to stop it — leak
```

✓ Cleans Up Properly

```
function startPolling() {  
  const id = setInterval(() => {  
    fetchData();  
  }, 1000);  
  return () => clearInterval(id);  
}  
const stop = startPolling();  
stop(); // memory released
```

WHY IT MATTERS

Every `setInterval`/`setTimeout`/`addEventListener` you create should have a matching `clear`/`removeEventListener`. In component-based UIs (React, Vue) this belongs in the `cleanup/unmount` phase.

Practical Memory Optimization Techniques

- Declare variables in the smallest scope possible — let engine reclaim memory sooner when the block exits.
- Avoid creating functions/objects inside hot loops — they allocate on every iteration; hoist them outside when possible.
- Use primitive values over wrapper objects: prefer 'text' over new String('text') — wrappers add heap overhead.
- Null out large object references you're truly done with in long-lived scopes (e.g. large caches): cache = null.
- Prefer typed arrays (Int32Array, Float64Array) for large numeric datasets — far more memory-efficient than regular arrays.
- Use WeakMap/WeakSet for caches keyed by objects — entries are garbage collected automatically when the key object is no longer referenced elsewhere.

WeakMap: A Self-Cleaning Cache

Unlike Map, a WeakMap holds weak references to its keys — the GC can collect the key (and its cache entry) once nothing else references it.



```
const cache = new WeakMap();

function process(userObj) {
  if (cache.has(userObj)) return cache.get(userObj);
  const result = expensiveComputation(userObj);
  cache.set(userObj, result);
  return result;
}

let user = { id: 1 };
process(user);
user = null; // cache entry is now eligible for GC automatically
```

05

Operators

Arithmetic, comparison, logical, and the modern operators every JS dev should default to.

Categories of Operators in JavaScript

- Arithmetic: `+` `-` `*` `/` `%` `**` — perform math; note `+` also does string concatenation.
- Assignment: `=` `+=` `-=` `*=` `/=` `%=` `**=` — assign and update in one step.
- Comparison: `==` `!=` `===` `!==` `>` `<` `>=` `<=` — compare values.
- Logical: `&&` `||` `!` — combine/invert boolean logic, but also used for control flow.
- Modern operators: `??` (nullish coalescing), `?.` (optional chaining), `??=` `/` `||=` `&&=` (logical assignment).
- Ternary: `condition ? exprIfTrue : exprIfFalse` — a compact conditional expression.

Arithmetic Operators & Precision Pitfalls

- Standard: + - * / and remainder %.
- Exponentiation ** (ES2016) replaces Math.pow() for most cases.
- JS numbers are IEEE-754 doubles — decimal math is not exact (0.1 + 0.2 !== 0.3).
- Best practice: never compare floats with === for equality; use a tolerance (epsilon) check.
- Best practice: for currency, work in integer cents or use a decimal library — never raw floats.

```
console.log(0.1 + 0.2);           //  
0.30000000000000004  
console.log(0.1 + 0.2 === 0.3);   // false  
  
function nearlyEqual(a, b, eps = 1e-9) {  
  return Math.abs(a - b) < eps;  
}  
nearlyEqual(0.1 + 0.2, 0.3); // true
```

== vs === — Strict Equality Is the Rule

- == (loose equality) performs type coercion before comparing — unpredictable results.
- === (strict equality) compares both value AND type — no coercion, predictable.
- The only conventional exception: value == null (checks for both null and undefined at once).
- Best practice: enable an ESLint rule (eqeqeq) to forbid == across the codebase.
- Same logic applies to !== over !=.

```
'5' == 5      // true  (coerced)
'5' === 5     // false (different types)
null == undefined // true
null === undefined // false

// The one accepted use of ==
if (value == null) { /* null OR undefined */ }
```

Logical Operators as Control Flow

- `&&` returns the first falsy value or the last value — used for conditional execution.
- `||` returns the first truthy value — classically used for default values (has a flaw: `0`/" fall through).
- `??` (nullish coalescing, ES2020) only falls back on null/undefined — not on `0`, `"`, or `false`.
- Best practice: prefer `??` over `||` whenever `0`, `"`, or `false` are valid values you want to keep.
- `!` inverts a boolean; `!!` is a common idiom to force-coerce any value to boolean.

```
const count = 0;
const a = count || 10;    // 10 — WRONG, 0 was
                           valid!
const b = count ?? 10;    // 0 — correct

user.isVerified && sendWelcomeEmail();
const isLoggedIn = !!sessionToken;
```


Optional Chaining & Logical Assignment

- `?.` safely accesses nested properties — short-circuits to undefined instead of throwing.
- Works on method calls too: `obj.method?.()` — skips the call if method doesn't exist.
- Logical assignment operators (ES2021) combine a logical check with assignment: `||=`, `&&=`, `??=`.
- Best practice: replace deep manual null-checks (`a && a.b && a.b.c`) with optional chaining.
- Best practice: use `??=` to lazily set a default only if a value is null/undefined.



```
const city = user?.address?.city ?? "Unknown";
user?.notify?.();

// Logical assignment
config.timeout ??= 3000; // set only if
null/undefined
settings.debug ||= false; // set only if falsy
flags.enabled &&= isValid; // set only if currently
truthy
```

Ternary vs If/Else — When to Use Which

✗ Overused Ternary

```
const msg = user.age >= 18
  ? user.hasID
    ? "Entry allowed"
    : "ID required"
  : "Underage";
// nested ternaries hurt readability
```

✓ Readable Choice

```
let msg;
if (user.age < 18) {
  msg = "Underage";
} else if (!user.hasID) {
  msg = "ID required";
} else {
  msg = "Entry allowed";
}
```

WHY IT MATTERS

Ternaries are great for a single, simple condition assigned to a variable or used inline in JSX/templates. Nesting them sacrifices readability for brevity — switch to if/else or a lookup object once you have more than one branch.

Operators — Quick Reference Card

===

Always prefer strict equality over `==` to avoid coercion surprises

??

Use for real defaults; keeps `0`, `"`, `false` as valid values (unlike `||`)

?.

Safely reads nested properties without manual null-chains

06

Loops & Iteration

Every looping construct in JS, when to use each, and their memory/performance profile.

Why So Many Loop Types Exist

- Every loop repeats a block of code — the differences are about control, readability, and what you're iterating over.
- Classic loops (for, while, do-while) give full manual control over the iteration variable.
- for...of iterates VALUES of any iterable (arrays, strings, Maps, Sets).
- for...in iterates ENUMERABLE KEYS of an object (including inherited ones) — rarely what you want for arrays.
- Array iteration methods (forEach, map, filter, reduce) are declarative, functional-style alternatives.
- Choosing the right one is a readability and correctness decision, not just style.

The Classic for Loop

- Syntax: `for (init; condition; increment) { }`.
- Best when you need the index, need to skip elements, or need multiple counters.
- The loop variable should be declared with `let` (block-scoped) — never `var` (see Hour 2).
- Fastest raw-iteration option in most engines because there's no per-item function call overhead.
- Best practice: cache `array.length` in the condition for very large arrays if it's expensive to recompute (rarely needed for plain arrays, but matters for live `NodeLists`).



```
const items = ["a", "b", "c", "d"];
for (let i = 0; i < items.length; i++) {
  console.log(i, items[i]);
}

// Reverse iteration
for (let i = items.length - 1; i >= 0; i--) {
  console.log(items[i]);
}
```

while and do-while

- `while (condition) { }` — checks condition BEFORE each iteration; may run zero times.
- `do { } while (condition)` — checks condition AFTER each iteration; always runs at least once.
- Best for when the number of iterations isn't known in advance (e.g. reading a stream until done).
- Always ensure the condition eventually becomes false — an infinite loop will freeze a single-threaded JS environment.
- Best practice: prefer `for/for...of` when a counter is involved; reserve `while` for open-ended conditions.

```
let attempts = 0;
while (attempts < 3 && !success) {
  success = tryConnect();
  attempts++;
}

do {
  renderFrame();
} while (isAnimating); // always runs once
```

for...of vs for...in — Don't Mix Them Up

- for...of → values of an iterable (Array, String, Map, Set, NodeList). Use this for arrays.
- for...in → enumerable keys of an object, including inherited ones from the prototype chain.
- Using for...in on an array works but is discouraged — iterates keys as strings, includes inherited/custom props, and order isn't guaranteed.
- Best practice: for...of (or array methods) for arrays; for...in only for plain objects, ideally guarded with hasOwnProperty.



```
const arr = ["x", "y", "z"];
for (const value of arr) console.log(value); // x y
z

const user = { name: "Sam", age: 28 };
for (const key in user) {
  if (Object.hasOwn(user, key)) {
    console.log(key, user[key]);
  }
}
```


break, continue & Labeled Loops

- `break` exits the loop entirely — execution jumps to the first line after the loop.
- `continue` skips the rest of the current iteration and moves to the next one.
- Labeled statements (rare, but useful) let `break/continue` target an outer loop from within a nested one.
- Best practice: prefer restructuring nested loops into a function with an early return over labeled breaks — usually more readable.

break, continue, and Labels in Action



```
outer: for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (j === 1) continue;      // skip j === 1
    if (i === 2) break outer;   // exit BOTH loops
    console.log(i, j);
  }
}

// Prefer this instead when possible:
function findMatch(matrix, target) {
  for (const row of matrix) {
    for (const cell of row) {
      if (cell === target) return true; // early return
    }
  }
  return false;
}
```

07

Array Iteration Methods

forEach, map, filter, reduce — the functional, declarative way to loop.

forEach — Declarative Side Effects

- Calls a function once per element; always returns undefined.
- Cannot break early (use a for...of loop or .some()/.every() if you need to stop).
- Best used for side effects: logging, DOM updates, pushing into an external array.
- More readable than a manual for loop when the index isn't needed.
- Memory: creates one function call per element — slightly more overhead than a raw for loop.



```
const cart = [{ name: "Pen", price: 10 }, { name: "Bag", price: 500 }];
cart.forEach((item, index) => {
  console.log(`${index}: ${item.name} - ₹${item.price}`);
});
```

map — Transform Into a New Array

- Returns a NEW array of the same length — never mutates the original.
- Use when you need a transformed version of every element (pure transformation).
- Chains naturally with filter/reduce for pipeline-style data processing.
- Best practice: don't use map purely for side effects (that's what forEach is for) — always use its return value.
- Memory: allocates a brand-new array — for very large datasets processed once, a for loop can be more memory-efficient.



```
const prices = [10, 500, 250];  
const withTax = prices.map(p => p * 1.18);  
console.log(withTax);           // [11.8, 590, 295]  
console.log(prices);           // unchanged — [10, 500,  
250]
```

filter — Select a Subset

- Returns a NEW array containing only elements where the callback returns true.
- Never mutates the original array; returns an empty array if nothing matches (never null/undefined).
- Commonly chained with map: `filter(...).map(...)` — filter first to shrink the dataset before transforming.
- Best practice: keep the predicate function pure (no side effects) — it should only test, not modify.



```
const products = [{ name: "Pen", stock: 0 }, {  
  name: "Bag", stock: 12 }];  
const inStock = products.filter(p => p.stock > 0);  
console.log(inStock); // [{ name: "Bag", stock: 12 }]
```

reduce — Fold an Array Into One Value

- The most general iteration method — `forEach`, `map`, and `filter` can all be implemented with `reduce`.
- Signature: `array.reduce((accumulator, current) => {...}, initialValue)`.
- Always provide an `initialValue` — omitting it on an empty array throws a `TypeError`.
- Great for totals, grouping, flattening, building objects from arrays.
- Best practice: keep the reducer function pure and return the accumulator every iteration.



```
const order = [{ item: "Pen", qty: 3, price: 10 },  
  { item: "Bag", qty: 1, price: 500 }];  
const total = order.reduce((sum, o) => sum + o.qty  
  * o.price, 0);  
console.log(total); // 530  
  
// Grouping with reduce  
const grouped = ["a", "bb", "c", "dd"].reduce((acc, w)  
  => {  
    const key = w.length;  
    (acc[key] ??= []).push(w);  
    return acc;  
  }, {});
```

Classic for Loop vs Array Methods — Performance Reality

✗ Looks Clean, Costs More

```

// 1M elements, called repeatedly in a hot path
const total = hugeArray
  .filter(x => x.active)
  .map(x => x.value)
  .reduce((a, b) => a + b, 0);
// 3 full passes + 2 intermediate arrays

```

✓ Optimized Hot Path

```

let total = 0;
for (let i = 0; i < hugeArray.length; i++) {
  const x = hugeArray[i];
  if (x.active) total += x.value;
}
// 1 pass, zero intermediate allocations

```

WHY IT MATTERS

For typical UI code (hundreds/thousands of items), array methods are the right default — readable and fast enough. For hot paths over very large datasets (100k+ elements, animation loops, real-time data), a single for loop avoiding intermediate array allocations measurably reduces both time and GC pressure.

Loop Selection — Decision Checklist

- **Need index-based control or multiple counters?**

→ Use a classic for loop.

- **Iterating array/string/Map/Set values, no index needed?**

→ Use for...of, or forEach for simple side effects.

- **Transforming, selecting, or aggregating array data (not mutating in place)?**

→ Use map / filter / reduce — write it as a pipeline.

- **Iterating object own keys?**

→ Use Object.keys()/entries()/values() with for...of, not for...in.

- **Extremely large dataset in a performance-critical hot path?**

→ Benchmark; a plain for loop often wins by avoiding intermediate arrays.

Loops — Quick Reference Card

for

Full control, index access, fastest raw iteration for hot paths

of / in

for...of → values (arrays); for...in → keys (plain objects only)

map/filter/reduce

Declarative, pure, chainable — the default for data transformation

08

Lab & Wrap-Up

Live debugging exercise, key takeaways, and homework before Class 2.

Spot the Bugs — Group Exercise (15 min)

This snippet has 5 issues covering everything from today: scoping, coercion, loops, and memory. Find them before we review together.

```
var results = [];  
for (var i = 0; i < 5; i++) {  
  setTimeout(function () {  
    results.push(i);  
  }, 100);  
}  
  
function getDiscount(price) {  
  if (price == "100") return price * 0.1;  
}  
  
let cache = {};  
for (key in someObject) {  
  cache[key] = someObject[key];  
}
```

Key Takeaways From Today

- Default to `const`, use `let` only when reassigning, and never write `var` in new code.
- Primitives copy by value; objects copy by reference — know which one you're handling.
- The GC handles memory for you, but leaks still happen through timers, listeners, and stray globals — clean up what you create.
- Always use `===` and `??` over `==` and `||` for predictable comparisons and defaults.
- Pick loops for readability first (`for...of`, `map/filter/reduce`); reach for a raw `for` loop only in proven hot paths.

Homework Before Class 2

- Rewrite 3 old var-based scripts of yours using let/const and fix any scope leaks you find.
- Implement a shopping cart total using reduce, then benchmark it against a for loop on 100k items.
- Read up on 'closures' — Class 2 builds directly on today's scope and memory concepts.
- Bring one real memory-leak bug you've hit (or found in the wild) to discuss.

6-PART JAVASCRIPT SERIES · SESSION 2 OF 6 · 8 HOURS

JavaScript Class 2

Functions & Scope — closures, higher-order functions and this

{ } JavaScript

Function Declarations vs. Expressions

- Definition — a function is a reusable, named block of code that performs a task or returns a value when called.
- When to use — anytime logic is repeated, needs a clear name, or should be organised into a testable, reusable unit.
- Basic syntax — `function functionName(parameters) { /* code */ }` — defines a named, reusable block of code.
- Declaration — `function greet() {}` — is fully hoisted; callable before its definition line.
- Expression — `const greet = function () {}` — only the variable binding hoists (TDZ for `let/const`), not the function body.
- Named function expressions aid stack traces and allow self-recursion without leaking the name outward.
- Declarations suit top-level utilities; expressions shine when assigning functions conditionally or passing them as values.
- Both create a new scope and close over their defining environment — today builds directly on Class 1's scoping rules.

```
// Declaration — hoisted
greet(); // works!
function greet() {
  console.log("Hi!");
}

// Expression — not hoisted
sayBye(); // TypeError
const sayBye = function () {
  console.log("Bye!");
};
```


Arrow Functions — Concise but Different

- Definition — a concise, ES6 syntax for writing function expressions using `=>`, e.g. `const add = (a, b) => a + b`.
- No own `this`, `arguments`, or `super` — arrow functions inherit these lexically from the enclosing scope.
- Cannot be used as constructors — `new Arrow()` throws a `TypeError`.
- Implicit return with a single expression: `const double = n => n * 2`;
- Ideal for callbacks (`map/filter/reduce`, event handlers) where you want `this` to stay bound to the outer context.
- Not a drop-in replacement for methods — avoid arrow functions for object methods that need their own `this`.

```
const team = {
  name: "Devs",
  regular: function () {
    console.log(this.name); // "Devs"
  },
  arrow: () => {
    console.log(this.name); // undefined
  }
};

team.regular(); // "Devs"
team.arrow();   // undefined
```

Closures — What They Are & Why They Matter

- A closure bundles a function with references to its surrounding (lexical) scope — created every time a function is defined.
- The inner function keeps access to outer variables even after the outer function has returned.
- This is how private state works in JS without classes — the counter/module pattern.
- Each call to the outer function creates a fresh, independent closure — separate memory, not shared.
- Closures are why Class 1's scope and memory model matter — captured variables stay alive, a common leak source if overused.

```
function makeCounter() {  
  let count = 0;  
  return function () {  
    count += 1;  
    return count;  
  };  
}  
  
const counter1 = makeCounter();  
counter1(); // 1  
counter1(); // 2  
  
const counter2 = makeCounter();  
counter2(); // 1 — independent state!
```

WHAT "CLOSURE" MEANS

A function that remembers the variables from where it was created, even after that outer scope has finished running.

Closures in Practice — Private State & Modules

- Closures let you fake 'private' fields — no class or # needed, just variables trapped in an outer function.
- The Revealing Module Pattern: return an object of methods that all close over the same private state.
- Memoisation: cache expensive results in a closure so repeat calls skip the work.
- One-time setup: run initialisation code once, then return a function that reuses the result.
- Trade-off: every live closure keeps its captured scope in memory — release references you no longer need.

```
function createBankAccount(balance) {  
  return {  
    deposit(amt) { balance += amt; return balance;  
  },  
    withdraw(amt) { balance -= amt; return balance;  
  },  
    getBalance() { return balance; }  
  };  
}  
  
const acct = createBankAccount(100);  
acct.deposit(50);    // 150  
acct.withdraw(30);   // 120  
acct.balance;        // undefined — private!
```

The Classic Loop-Closure Bug

- `var` is function-scoped, so every `setTimeout` callback in a `for (var...) loop` closes over the same `i`.
- By the time the callbacks run, the loop has already finished and `i` holds its final value.
- `let` creates a fresh binding for `i` on every iteration, so each closure captures its own snapshot.
- Legacy fix for `var`: wrap the loop body in an IIFE to create a new scope per iteration.
- Rule of thumb: use `let/const` in loops with async callbacks; keep `var` only for legacy code.

```
// The bug
for (var i = 1; i <= 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// logs: 4, 4, 4

// The fix
for (let i = 1; i <= 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// logs: 1, 2, 3
```

WHY IT HAPPENS

`var` has one binding for the whole loop; `let` creates a brand-new `i` each pass, so every closure captures a different value.

Higher-Order Functions — Functions as Values

- In JS, functions are first-class values — assign them to variables, pass them as arguments, return them from other functions.
- A higher-order function either accepts a function as a parameter, returns one, or both.
- Callbacks let you inject custom behaviour into generic code — what happens after a click, a fetch, or each array item.
- Returning a function lets you build customised functions on demand, with configuration baked in via closure.
- This is the foundation for map/filter/reduce, event handlers, and Promise .then() chains.

```
function greetWith(greeting) {  
  return function (name) {  
    console.log(`${greeting}, ${name}!`);  
  };  
}  
  
const greetFormally = greetWith("Good day");  
greetFormally("Ms. Lee");  
// "Good day, Ms. Lee!"  
  
function process(items, callback) {  
  for (const item of items) callback(item);  
}  
process([1, 2, 3], n => console.log(n * 2));
```

Array HOFs Deep Dive — map, filter, reduce

- `map()` transforms every element and returns a new array of the same length — never mutates the original.
- `filter()` keeps only elements that pass a test, returning a new, possibly shorter array.
- `reduce()` folds the array down to a single value — a total, an object, even another array.
- All three take a callback of (element, index, array) and are chainable: `data.filter(...).map(...)`.
- Prefer these over manual for-loops when transforming data — they're declarative and easier to read and test.



```
const prices = [12, 45, 8, 20];

const withTax = prices.map(p => p * 1.08);
// [12.96, 48.6, 8.64, 21.6]

const affordable = prices.filter(p => p < 20);
// [12, 8]

const total = prices.reduce((sum, p) => sum + p, 0);
// 85

const summary = prices
  .filter(p => p < 20)
  .reduce((s, p) => s + p, 0); // 20
```

The this Keyword — Rules Across Contexts

- this isn't fixed by where a function is defined — it's set by how the function is called, its “call-site”.
- Plain function call: this is undefined in strict mode (or the global object in sloppy mode).
- Method call — obj.method(): this is the object left of the dot at call time.
- Arrow functions have no this of their own — they inherit it from the enclosing lexical scope.
- Constructor calls (new Fn()) and explicit binding (call/apply/bind) override the default rules entirely.

```
const user = {
  name: "Ada",
  greet() { console.log(this.name); },
  greetLater: () => console.log(this.name)
};

user.greet();           // "Ada" — this = user

const fn = user.greet;
fn();                   // undefined — lost object

user.greetLater();     // undefined — arrow
                        // uses outer this
```

Explicit Binding — call, apply, bind

- `call(thisArg, a, b, ...)` invokes the function immediately with this set to `thisArg`, arguments listed individually.
- `apply(thisArg, [a, b, ...])` does the same, but arguments are passed as a single array.
- `bind(thisArg)` returns a brand-new function permanently bound to `thisArg` — it doesn't run until you call it.
- Common uses: borrowing array methods on array-like objects, and fixing this in callbacks or event handlers.
- Arrow functions ignore `call/apply/bind` entirely — their `this` is locked to the enclosing scope for good.



```
function introduce(greeting) {  
  console.log(`${greeting}, I'm ${this.name}`);  
}  
  
const person = { name: "Priya" };  
  
introduce.call(person, "Hi");  
introduce.apply(person, ["Hi"]);  
  
const bound = introduce.bind(person);  
bound("Hello"); // runs later, this fixed
```


Homework Before Class 3

- Refactor one of your own functions into an arrow function and note what changes — this, arguments, hoisting.
- Write a memoize(fn) helper using a closure — cache results so repeat calls with the same input skip the work.
- Rebuild a for-loop you've written before using map/filter/reduce instead, and compare the readability.
- Fix the classic var-in-a-loop bug in a small script by switching to let, then verify the output changes.
- Read up on 'the event loop' and asynchronous JavaScript — Class 3 builds directly on today's functions and callbacks.